

Project 4 — Resume Uploader

A basic-style project, but the key focus areas here are: - Setting up **Alembic with MySQL** (via phpMyAdmin — viewing the database at localhost) - Handling **form fields** and file uploads properly

Step 1: Create Virtual Environment

Create and activate a virtual environment.

Step 2: Install Dependencies

Install the following packages inside the virtual environment:

```
pip install "fastapi[standard]" "sqlalchemy[asyncio]" alembic python-decouple aiomysql
```

Step 3: Freeze Requirements & Take Notes

```
pip freeze > requirements.txt
```

- Make a notes file and write down all the commands run, step by step.

Initialize Alembic (async template):

```
alembic init -t async alembic
```

Make sure the **XAMPP server** is ON.

Step 4: Create `.env` File

Create `.env` inside the root folder: `ch119/.env`

Here we write the database credentials:

```
DB_USER = raj
DB_PASS = raj123
DB_NAME = resume
DB_PORT = 3306
DB_USER = localhost
```

Setting up the MySQL user in XAMPP / phpMyAdmin

1. Start XAMPP → go to `localhost` → open **phpMyAdmin**
2. Go to **User accounts** → **Create new user**
3. Use the **same username and password** as in `.env`
4. Check **all Global privileges** → click **Go**
5. The new user is now created
6. Go into the database section and **create the `resume` database**

Step 5: Project Folder Structure

Go into `ch119/` and set up the following structure:

- `ch119/app/__init__.py`
- `ch119/app/main.py`
- `ch119/app/db/__init__.py`
- `ch119/app/db/config.py`

`config.py`

```

from sqlalchemy.ext.asyncio import create_async_engine, async_sessionmaker, AsyncSession, AsyncAttrs
from sqlalchemy.orm import DeclarativeBase
from fastapi import Depends
from typing import AsyncGenerator, Annotated
from decouple import config

# Here using the decouple to config for the mysql phpmyadmin
DB_USER = config("DB_USER")
DB_PASS = config("DB_PASS")
DB_NAME = config("DB_NAME")
DB_PORT = config("DB_PORT", cast=int)
DB_HOST = config("DB_HOST")

# Db roots
DATABASE_URL = f"mysql+aiomysql://{DB_USER}:{DB_PASS}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
engine = create_async_engine(DATABASE_URL, echo=True, future=True)
async_session = async_sessionmaker(bind=engine, expire_on_commit=False, class_=AsyncSession)

class Base(AsyncAttrs, DeclarativeBase):
    pass

async def get_session() -> AsyncGenerator[AsyncSession, None]:
    async with async_session() as session:
        yield session

SessionDep = Annotated[AsyncSession, Depends(get_session)]

```

Step 6: Configure Alembic

`alembic.ini`

Find this line:

```
sqlalchemy.url = driver://user:pass@localhost/dbname
```

Change it to:

```
sqlalchemy.url = mysql+aiomysql://user:password@localhost:3306/dbname
```

`alembic/env.py`

First, import `Base` and `decouple`:

```

from app.db.config import Base
from decouple import config as env_config

```

Load the `DATABASE_URL` dynamically from `.env`:

```

# --- Load DATABASE_URL dynamically from .env ---
DB_USER = env_config("DB_USER")
DB_PASS = env_config("DB_PASS")
DB_NAME = env_config("DB_NAME")
DB_PORT = env_config("DB_PORT", cast=int)
DB_HOST = env_config("DB_HOST")

```

Then build the URL:

```
DATABASE_URL = f"mysql+aiomysql://{DB_USER}:{DB_PASS}@{DB_HOST}:{DB_PORT}/{DB_NAME}"
```

Set it as the main option:

```
config.set_main_option("sqlalchemy.url", DATABASE_URL)
```

Find this:

```
target_metadata = Base.metadata
```

Change into:

```
target_metadata = Base.metadata
```

After that, in `alembic/env.py`, find:

```
url = config.get_main_option("sqlalchemy.url")
context.configure(
    url=url,
    target_metadata=target_metadata,
    literal_binds=True,
    dialect_opts={"paramstyle": "named"},
    compare_type=True,
)
```

Add `compare_type=True` (as shown above).

Also find:

```
def do_run_migrations(connection: Connection) -> None:
    context.configure(connection=connection, target_metadata=target_metadata)
```

And also add `compare_type=True` here:

```
def do_run_migrations(connection: Connection) -> None:
    context.configure(connection=connection, target_metadata=target_metadata, compare_type=True,)
```

✓ **Alembic setup done** — we just need to import each model here as we create it.

Step 7: Create the Resume Module & Models

Create folder: `ch119/app/resume/` containing: - `__init__.py` - `models.py` - `services.py` - `schemas.py` - `utils.py` - `routers.py`

`models.py`

```
from sqlalchemy.orm import Mapped, mapped_column
from sqlalchemy import String, Text
from app.db.config import Base
from datetime import date
import enum

class GenderEnum(str, enum.Enum):
    male = "male"
    female = "female"
    other = "other"

class Resume(Base):
    __tablename__ = "resumes"

    id: Mapped[int] = mapped_column(primary_key=True, index=True)
    name: Mapped[str] = mapped_column(String(255))
    email: Mapped[str] = mapped_column(String(255), unique=True)
    dob: Mapped[date]
    state: Mapped[str] = mapped_column(String(200))
    gender: Mapped[GenderEnum]
    preferred_locations: Mapped[str] = mapped_column(Text)
    image_path: Mapped[str] = mapped_column(String(255))
    resume_file_path: Mapped[str] = mapped_column(String(255))
```

⚠ **Important:** After writing this model, don't forget to import it in `alembic/env.py`:

```
from app.resume import models as resumemodel
```

Generate the migration

```
alembic revision --autogenerate -m "create table"
```

This command generates the version file — check it inside `ch119/alembic/versions/` (make changes there if needed).

Then run:

```
alembic upgrade head
```

Step 8: Schemas & Utils

ch119/app/resume/schemas.py

```
from pydantic import BaseModel, EmailStr
from datetime import date
from typing import List
from enum import Enum

class GenderEnum(str, Enum):
    male = "male"
    female = "female"
    other = "other"

class ResumeCreate(BaseModel):
    name: str
    email: EmailStr
    dob: date
    state: str
    gender: GenderEnum
    preferred_locations: List[str]

class ResumeOut(BaseModel):
    id: int
    name: str
    email: EmailStr
    dob: date
    state: str
    gender: GenderEnum
    preferred_locations: str
    image_path: str
    resume_file_path: str
    model_config = {
        "from_attributes": True
    }
```

ch119/app/resume/utils.py

We need to create an `uploads` folder at the root directory (`ch119/uploads`) — this is where uploaded files will be stored.

```
import os
from uuid import uuid4
from fastapi import UploadFile

UPLOAD_DIR = "uploads"
os.makedirs(UPLOAD_DIR, exist_ok=True)

async def save_upload_file(upload_file: UploadFile, sub_dir: str) -> str:
    ext = os.path.splitext(upload_file.filename)[-1]
    filename = f"{uuid4().hex}{ext}"
    dir_path = os.path.join(UPLOAD_DIR, sub_dir)
    os.makedirs(dir_path, exist_ok=True)
    file_path = os.path.join(dir_path, filename)
    with open(file_path, "wb") as f:
        content = await upload_file.read()
        f.write(content)
    return file_path
```

Step 9: Services & Routers

services.py

```
from fastapi import HTTPException, UploadFile
from app.resume.models import Resume
from app.resume.schemas import ResumeCreate
from sqlalchemy.ext.asyncio import AsyncSession
from sqlalchemy import select
from app.resume.utils import save_upload_file

async def create_resume(session: AsyncSession, data: ResumeCreate, image: UploadFile, resume_file: UploadFile):
    stmt = select(Resume).where(Resume.email == data.email)
    result = await session.scalars(stmt)
    if result.first():
        raise HTTPException(status_code=400, detail="Email already exist")
    # Save uploaded files
    image_path = await save_upload_file(image, "images")
    file_path = await save_upload_file(resume_file, "files")
    resume = Resume(
        name=data.name,
```

```

        email=data.email,
        dob=data.dob,
        state=data.state,
        gender=data.gender,
        preferred_locations=",".join(data.preferred_locations),
        image_path=image_path,
        resume_file_path=file_path
    )
    session.add(resume)
    await session.commit()
    await session.refresh(resume)
    return resume

async def get_all_resumes(session: AsyncSession):
    resumes = await session.scalars(select(Resume))
    return resumes.all()

```

outers.py

```

from typing import List
from fastapi import APIRouter, UploadFile, Form, File
from app.db.config import SessionDep
from app.resume.schemas import ResumeCreate, ResumeOut, GenderEnum
from app.resume.services import create_resume, get_all_resumes

router = APIRouter(prefix="/resumes", tags=["Resume"])

@router.post("/upload", response_model=ResumeOut)
async def upload_resume(
    *,
    name: str = Form(...),
    email: str = Form(...),
    dob: str = Form(...),
    state: str = Form(...),
    gender: GenderEnum = Form(...),
    preferred_locations: List[str] = Form(...),
    image: UploadFile = File(...),
    resume_file: UploadFile = File(...),
    session: SessionDep
):
    data = ResumeCreate(
        name=name,
        email=email,
        dob=dob,
        state=state,
        gender=gender,
        preferred_locations=preferred_locations,
    )
    return await create_resume(session, data, image, resume_file)

@router.get("/", response_model=List[ResumeOut])
async def list_resumes(session: SessionDep):
    resumes = await get_all_resumes(session)
    return resumes

```

ch119/app/main.py

```

from fastapi import FastAPI
from app.resume.routers import router as resume_router

app = FastAPI()
app.include_router(resume_router)

```

Step 10:

(To be continued — instructor's next step not yet covered.)